

Problem 1

- Need to minimize the following:

$$C(\boldsymbol{\omega}) = -\frac{1}{N}L(\boldsymbol{\omega}) + \lambda \boldsymbol{\omega}^T \boldsymbol{\omega}$$

where

$$L(\boldsymbol{\omega}) = \sum_{j=1}^n y_j(\omega_0 + \sum_{i=1}^p \omega_i x_{ji}) - \ln(1 + \exp(\omega_0 + \sum_{i=1}^p \omega_i x_{ji}))$$

We minimize this loss using gradient descent, using:

$$\boldsymbol{\omega}^{t+1} \leftarrow \boldsymbol{\omega}^t - \eta \lambda \boldsymbol{\omega}^t + \frac{\eta}{N} \frac{\partial L(\boldsymbol{\omega}^t)}{\partial \boldsymbol{\omega}}$$

Let the linear predictor for observation j be

$$z_j = w_0 + \sum_{i=1}^p w_i x_{ji},$$

and define the augmented covariate vector $\tilde{\mathbf{x}}_j = (1, x_{j1}, \dots, x_{jp})^T$ and the parameter vector $\boldsymbol{\omega} = (w_0, w_1, \dots, w_p)^T$. The log-likelihood can then be written as

$$L(\boldsymbol{\omega}) = \sum_{j=1}^N [y_j z_j - \ln(1 + e^{z_j})].$$

Since $\partial z_j / \partial \boldsymbol{\omega} = \tilde{\mathbf{x}}_j$, applying the chain rule yields

$$\frac{\partial L(\boldsymbol{\omega})}{\partial \boldsymbol{\omega}} = \sum_{j=1}^N \left(y_j - \frac{e^{z_j}}{1 + e^{z_j}} \right) \tilde{\mathbf{x}}_j.$$

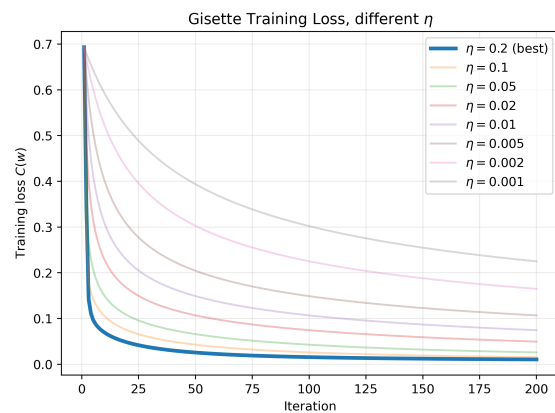
Stacking terms gives the gradient in vector form,

$$\nabla_{\boldsymbol{\omega}} L(\boldsymbol{\omega}) = \tilde{\mathbf{X}}^T \left(\mathbf{y} - \frac{e^{\mathbf{z}}}{1 + e^{\mathbf{z}}} \right),$$

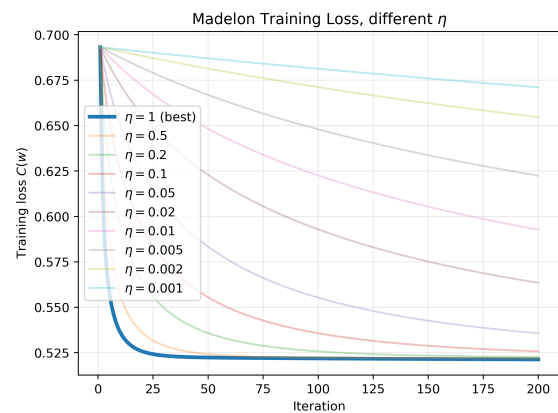
where $\tilde{\mathbf{X}}$ is the augmented design matrix with rows $\tilde{\mathbf{x}}_j^T$ and $\mathbf{z} = \tilde{\mathbf{X}} \boldsymbol{\omega}$.

For each of the three datasets, we loop over a grid to find the best η (in terms of smallest final training loss), and obtain the plots shown in figure 1. We then report the misclassification error rates for the train and test datasets for the three datasets. This is reported in 1.

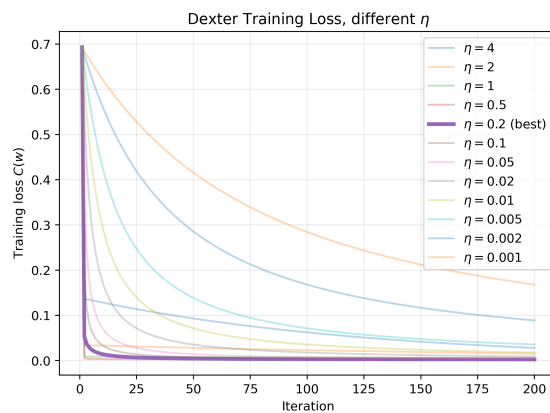
Lastly, we are asked to plot the ROC curve of both the training and test sets. We do so in figure 2



(a) Gisette: training loss vs. iteration for multiple η values.



(b) Madelon: training loss vs. iteration for multiple η values.

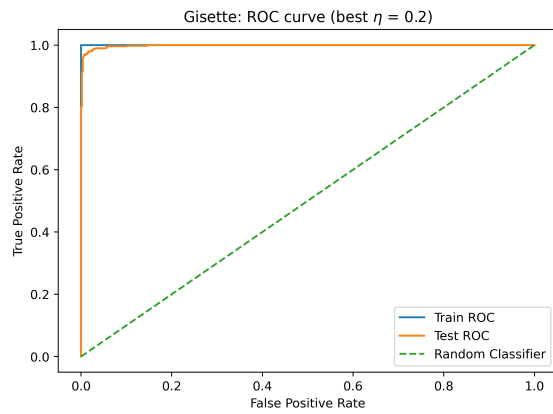


(c) Dexter: training loss vs. iteration for multiple η values.

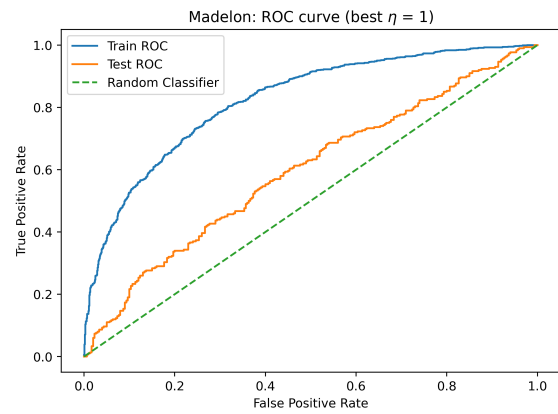
Figure 1: Training losses vs. iteration for logistic regression across possible learning rates η .

dataset	best eta	train misclass	test misclass
Gisette	0.200000	0.000000	0.020000
Madelon	1.000000	0.259500	0.426667
Dexter	0.200000	0.000000	0.140000

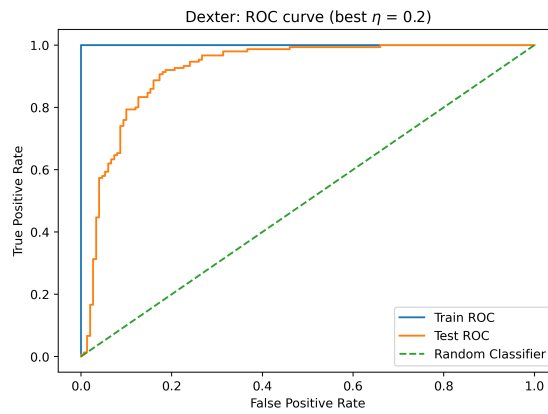
Table 1: Best learning rate η , training and test misclassification error, Logistic Regression.



(a) Gisette: training and test ROC curves for best η .



(b) Madelon: training and test ROC curves for best η .



(c) Dexter: training and test ROC curves for best η .

Figure 2: ROC curves for logistic regression on the training and test sets for each dataset.

Problem 2

We need to minimize analytically:

$$C(\boldsymbol{\omega}) = \frac{1}{N} \sum_{i=1}^n (y_i - \boldsymbol{\omega}^T \mathbf{x}_i - \omega_0)^2 + \lambda (\boldsymbol{\omega}^T \boldsymbol{\omega} + \omega_0^2)$$

where $\lambda = 0.001$, $(\mathbf{x}_i, y_i)$ are the training data, and $y_i \in \{-1, +1\}$. Predict classification via $\hat{y} = \text{sign}(\boldsymbol{\omega}^T \mathbf{x} + \omega_0)$.

If we define the augmented covariate vector and parameter vector as in question 1, then we can write the minimization function as

$$C(\tilde{\boldsymbol{\omega}}) = \frac{1}{N} \|\mathbf{y} - \tilde{\mathbf{X}} \tilde{\boldsymbol{\omega}}\|^2 + \lambda \|\tilde{\boldsymbol{\omega}}\|^2.$$

$$C(\boldsymbol{\omega}) = \frac{1}{N} \left(\mathbf{y}^T \mathbf{y} - 2\mathbf{y}^T \tilde{\mathbf{X}} \tilde{\boldsymbol{\omega}} + \tilde{\boldsymbol{\omega}}^T \tilde{\mathbf{X}}^T \tilde{\mathbf{X}} \tilde{\boldsymbol{\omega}} \right) + \lambda \tilde{\boldsymbol{\omega}}^T \tilde{\boldsymbol{\omega}}$$

We can take derivatives with respect to $\tilde{\boldsymbol{\omega}}$ and setting equal to 0:

$$\nabla_{\boldsymbol{\omega}} C(\boldsymbol{\omega}) = \frac{1}{N} \left(-2\tilde{\mathbf{X}}^T \mathbf{y} + 2\tilde{\mathbf{X}}^T \tilde{\mathbf{X}} \tilde{\boldsymbol{\omega}} \right) + 2\lambda \tilde{\boldsymbol{\omega}}$$

$$0 = 2 \left(\frac{1}{N} \left(\tilde{\mathbf{X}}^T \tilde{\mathbf{X}} \tilde{\boldsymbol{\omega}} - \tilde{\mathbf{X}}^T \mathbf{y} \right) + \lambda \tilde{\boldsymbol{\omega}} \right)$$

$$0 = \frac{1}{N} \left(\tilde{\mathbf{X}}^T \tilde{\mathbf{X}} \tilde{\boldsymbol{\omega}} - \tilde{\mathbf{X}}^T \mathbf{y} \right) + \lambda \tilde{\boldsymbol{\omega}}$$

$$0 = \tilde{\mathbf{X}}^T \tilde{\mathbf{X}} \tilde{\boldsymbol{\omega}} - \tilde{\mathbf{X}}^T \mathbf{y} + \lambda N \tilde{\boldsymbol{\omega}}$$

$$\left(\tilde{\mathbf{X}}^T \tilde{\mathbf{X}} + \lambda N I \right) \tilde{\boldsymbol{\omega}} = \tilde{\mathbf{X}}^T \mathbf{y}$$

where I is the $(p+1) \times (p+1)$ identity matrix (for matrix multiplication). We can use Python linear algebra solvers to find a solution to this (and we do), but the analytic solution would be:

$$\hat{\tilde{\boldsymbol{\omega}}} = \left(\tilde{\mathbf{X}}^T \tilde{\mathbf{X}} + \lambda N I \right)^{-1} \tilde{\mathbf{X}}^T \mathbf{y}$$

- We can report the misclassification error of the training and test datasets in the following table:

train misclass error	test misclass error
0.000000	0.091000

Table 2: misclassification Error for training and test datasets, Ridge like regression

The train misclassification error rate is the same (0.00), though the test error rate is higher (0.091 vs. 0.020).

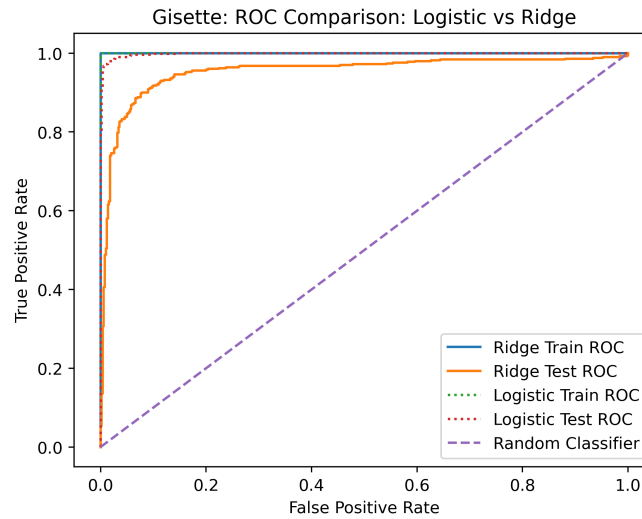


Figure 3: abcd.

- We can display the ROC curve for both the ridge and logistic models in figure 3.

The train ROC for the logistic and ridge models are practically the same. The ROC for the logistic model is better than the ridge though; tighter to the upper left hand corner.